

UNIT 2

ER MODEL

1. Database Design Phases

1.1 Initial Phase

- The initial phase of database design is to **characterize fully the data needs** of the prospective database users.
- The designer chooses a data model and translates these requirements into a **conceptual schema**.
- **Functional Requirements:** A fully developed conceptual schema indicates the functional requirements of the enterprise. In a “specification of functional requirements”, users describe the kinds of operations (or transactions) that will be performed on the data.

1.2 Final Design Phases

The process moves from an abstract data model to implementation in two phases:

1. **Logical Design:** Deciding on the database schema.
 - **Business decision:** What attributes should we record in the database?
 - **Computer Science decision:** What relation schemas should we have and how should the attributes be distributed among them?
2. **Physical Design:** Deciding on the physical layout of the database.

2. The Entity-Relationship (ER) Model

2.1 Basic Concepts

The ER model views an enterprise as a collection of entities and relationships.

- **Entity:** A “thing” or “object” in the enterprise that is distinguishable from other objects (e.g., a specific person, company, event, plant).
- **Entity Set:** A set of entities of the **same type** that share the same properties (e.g., set of all persons, set of all companies).
- **Attributes:** Entities are described by a set of attributes (descriptive properties).
 - *Example:* `instructor = (ID, name, street, city, salary)`.

2.2 Types of Attributes

Attributes can be classified into several types:

- **Simple Attributes / Atomic Attributes:** Cannot be divided further.
 - **Composite Attributes:** Can be divided into sub-parts (e.g., `Name` divided into `First Name`, `Middle Initial`, `Last Name`).
 - **Single-valued Attributes:** Have a single value for a particular entity.
 - **Multi-valued Attributes:** Can have multiple values for a single entity (e.g., Phone numbers, Degrees, Skills).
 - **Derived Attributes:** Values that can be derived from other attributes (e.g., `Age` derived from `Date of Birth`).
 - **Stored Attributes:** Attributes whose values are physically stored in the database and used to derive other attributes.
Example: `Date_of_Birth` is stored and used to calculate `Age`.
 - **Null Value Attribute:** An optional attribute where the value is unknown or not applicable.
-

3. Keys

Keys allow us to distinguish entities from one another and identify relationships.

3.1 Types of Keys

1. Super Key:

- A set of one or more attributes that, taken collectively, allow us to **identify uniquely** an entity in the entity set.
- *Example:* `customer-id` is a superkey. The combination `(customer-name, customer-id)` is also a superkey.

2. Candidate Key:

- A candidate key is a **minimal super key**.
- It is a super key from which you **cannot remove any fields** to get the unique records.
- *Example:* If `(Roll_No)` is unique and `(Name, Class)` is unique, both are candidate keys.

3. Primary Key:

- **One** out of the candidate keys selected to be the main identifier.
- It **cannot be null**.
- *Note:* Only one primary key is allowed per relation.

4. Alternate Keys:

- The candidate keys which are **not selected** as Primary Keys are called Alternate keys.

Easy Example to Understand Keys

Consider a **STUDENT** entity with the following attributes:

STUDENT (Roll_No, Aadhaar_No, Email, Name)

Assume:

- Roll_No is unique
 - Aadhaar_No is unique
 - Email is unique
-

1. Super Keys

A **super key** is any set of attributes that can uniquely identify a student.

Examples of super keys:

- (Roll_No)
- (Aadhaar_No)
- (Email)
- (Roll_No, Name)
- (Aadhaar_No, Email)

Super keys may contain **extra attributes**.

2. Candidate Keys

A **candidate key** is a **minimal super key**, meaning no attribute can be removed.

Examples of candidate keys:

- (Roll_No)
- (Aadhaar_No)
- (Email)

These are the **smallest possible keys** that still uniquely identify a student.

3. Primary Key

The **primary key** is **one candidate key chosen** to identify records.

Example:

- Primary Key: (Roll_No)

Only **one primary key** is allowed, and it **cannot be NULL**.

4. Alternate Keys

The **candidate keys not chosen** as the primary key are called **alternate keys**.

Examples:

- (Aadhaar_No)
 - (Email)
-

3.2 Foreign Key (Referential Integrity)

- **Definition:** Foreign Key constraints designate a column or combination of columns as a foreign key. It links back to the **primary key** (or a unique key) in another table.
 - **Terminology:**
 - **Child Table:** The table containing the foreign key.
 - **Parent Table:** The table containing the referenced key.
 - **Constraint Rule:** To satisfy a referential-integrity constraint, a foreign-key value must match an existing value in the parent table or be **NULL**.
 - **Order of Creation:** You must first have a **parent primary key defined** before you can create a foreign key in a child table.
-

4. Relationships

4.1 Definition and Degree

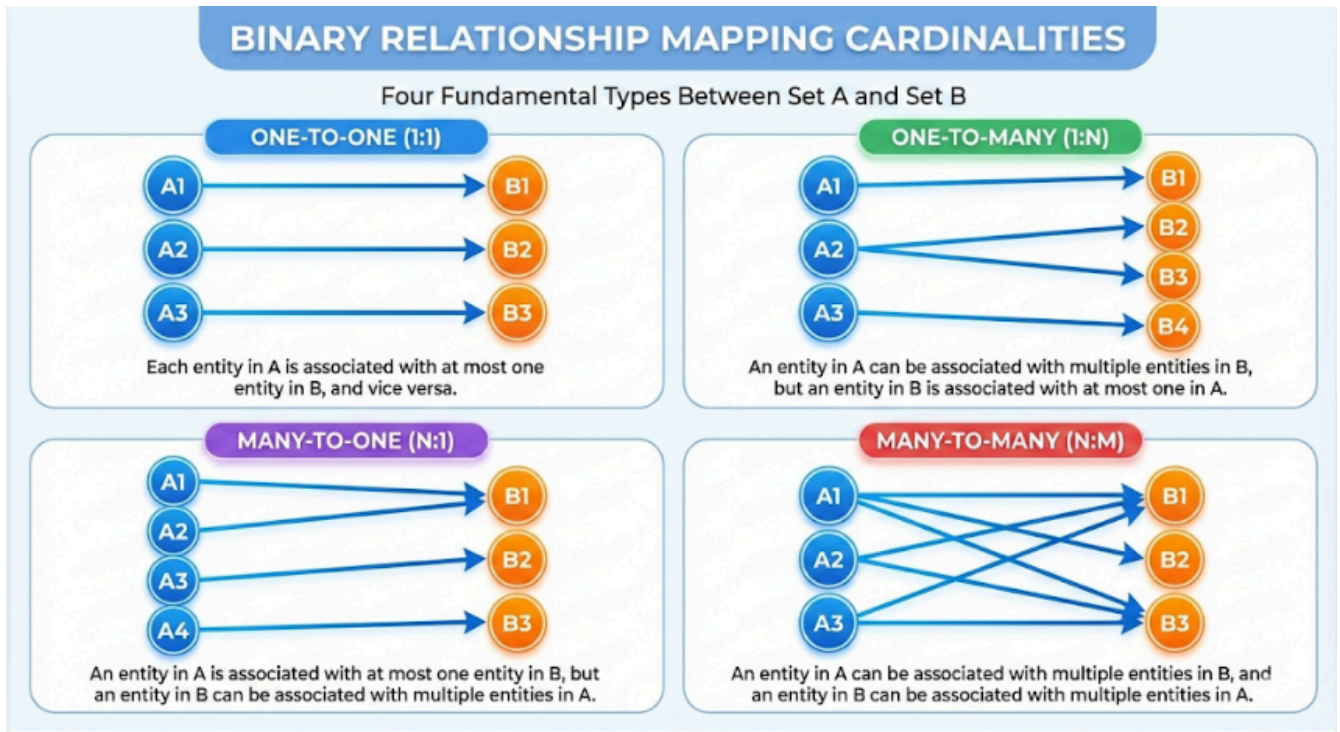
- **Relationship:** An association among several entities.
- **Relationship Set:** A set of relationships of the same type.
- **Degree:**
 - **Binary Relationship:** Involves **two** entity sets. Most relationship sets in a database system are binary.
 - **Ternary Relationship:** Involves **three** entity sets. Relationships between more than two entity sets are rare.
- **Attributes on Relationships:** An attribute can be associated with a relationship set.
 - *Example:* The `advisor` relationship between `instructor` and `student` may have the attribute `date`.

4.2 Mapping Cardinalities

Expresses the number of entities to which another entity can be associated via a relationship set.

1. **One to One**
2. **One to Many**
3. **Many to One**
4. **Many to Many**

- *Note:* Some elements in sets A or B may not be mapped to any elements in the other set.



5. ER Diagram Notation

ER diagrams express the overall logical structure of a database graphically.

5.1 Symbols

- **Rectangles:** Represent **Entity Sets**.
- **Ellipses:** Represent **Attributes**.
- **Diamonds:** Represent **Relationship Sets**.
- **Lines:** Link attributes to entity sets and entity sets to relationship sets.
- **Double Ellipses:** Represent **Multivalued attributes**.
- **Dashed Ellipses:** Denote **Derived attributes**.
- **Double Rectangles:** Represent **Weak Entity Sets**.

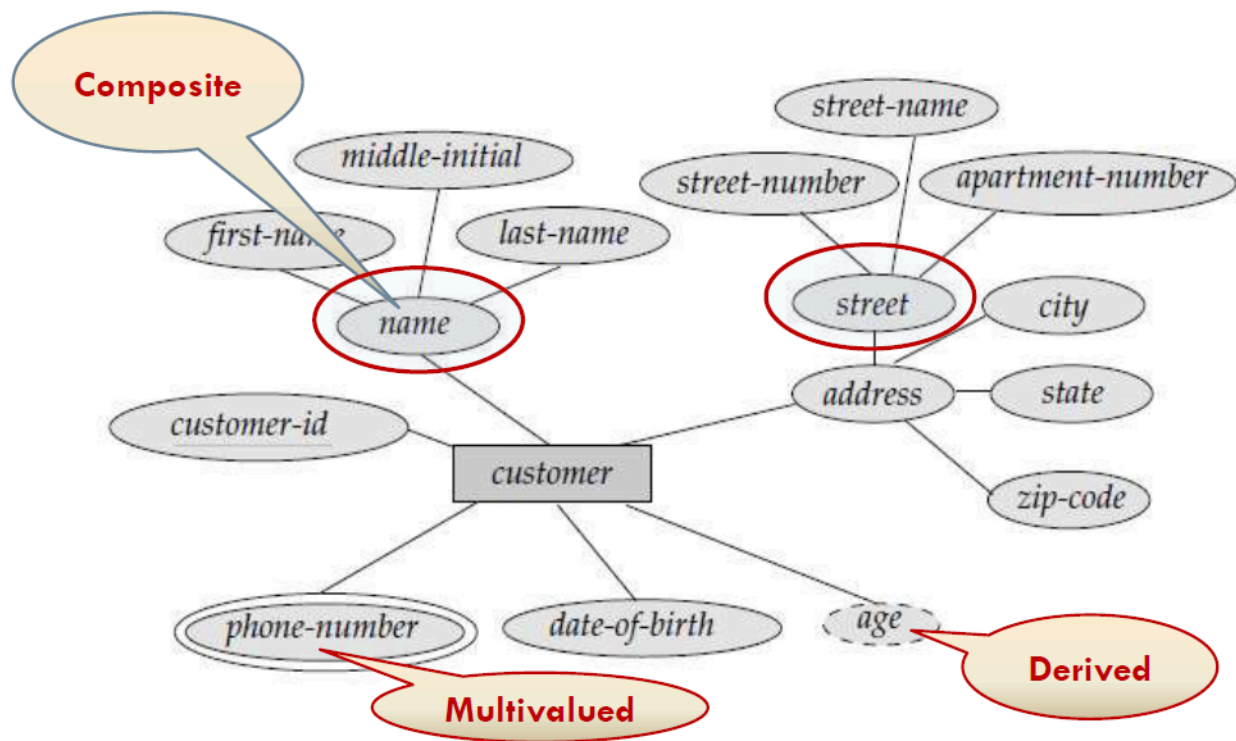
- **Double Lines:** Indicate **Total Participation** of an entity in a relationship set.

ER DIAGRAM SYMBOL REFERENCE		
Diagram Symbol	Symbol	One line description
	Entity set	Represents a collection of similar entities (e.g., STUDENT, COURSE).
	Weak entity set	An entity that cannot be uniquely identified by its own attributes (e.g., ENROLLMENT).
	Relationship	Represents an association between two or more entities (e.g., ENROLLS).
	Identifying relationship	Connects a weak entity to its owner entity (e.g., between STUDENT and ENROLLMENT).
	Attribute	A property or characteristic of an entity (e.g., Name, Credits).
	Multivalued attribute	An attribute that can have multiple values for a single entity (e.g., Phone_Number).
	Derived attribute	An attribute whose value is calculated from other attributes (e.g., Age).
	Primary key	An attribute that uniquely identifies each entity in an entity set (e.g., Student_ID).
	Discriminating attribute	Used to distinguish weak entities within the same owner entity (e.g., Enroll_No).
	Total participation	Indicates that every entity must participate in the relationship (e.g., ENROLLMENT side).

5.2 Weak Entity Sets

- **Definition:** An entity set that **does not have sufficient attributes** to form a primary key.
- **Existence:** A weak entity set is **existence dependent** on another entity, called the **identifying entity** (or owner).
- **Discriminator (Partial Key):** A set of attributes that allows distinction within the weak entity set.
 - *Formation:* The Primary Key of a weak entity = (PK of Identifying Entity) + (Discriminator of Weak Entity).
- **Identifying Relationship:** The relationship associating the weak entity with the owner. It is **many-to-one** from the weak entity to the identifying entity, and participation is **total**.

Sample ER Diagram



Role Indicators

- The labels *manager* and *worker* are called roles they specify how employee entities interact via the *works_for* relationship set
- Role labels are optional, and are used to clarify semantics of the relationship

